

Set Counting, Modular Patterns, and Mathematical Induction

N-20220808

§1.1 A set-counting problem

Consider finite sets $A_1, A_2, \dots, A_{32}$ subject to prescribed intersection conditions. A useful toy model gives each set a fixed size and controls every pairwise intersection explicitly.

The general strategy is inclusion–exclusion. If every set has m elements and pairwise overlaps have controlled size, then the size of the union is not simply the sum of sizes: subtract pairwise overlaps and add back higher overlaps. The construction yields a count of the form

$$31 + 28 \cdot 30 = 871.$$

The exact model is less important than the idea: when many sets overlap, one should count by layers, not by naive addition.

§1.2 A modular arithmetic construction

Another problem constructs two sets:

$$A = \{1, 4, 6, 7, 9, \dots, 1998\},$$

and a reversed or complementary set near 2000. The note observes that the pattern is governed by residues modulo 11. Since

$$2000 = 11 \cdot 181 + 9,$$

one can write elements as

$$11t + r.$$

The allowed residues are something like

$$r \in \{1, 4, 6, 7, 9\}.$$

The real insight is that forbidden differences correspond to forbidden residue differences. Instead of checking hundreds of numbers, one checks a small table modulo 11. This is one of the most common olympiad uses of modular arithmetic.

§1.3 Why not every construction is finished

The page explicitly says “problem reserved” in a few places. That is an honest part of mathematical notes. The important thing is to record what was tried and why it seemed plausible. Here the attempt is to remove residues that conflict pairwise and leave a maximal collection. Whether the construction is optimal requires a separate upper-bound argument.

This distinction is crucial: constructing an example proves a lower bound, but proving maximality requires an upper bound.

§1.4 Sequence exercises

Arithmetic and geometric sequences, partial sums, and recurrence-like identities provide a second family of examples. One problem has

$$a_n = a_1 + (n - 1)d$$

and conditions such as

$$a_1 + a_2 + \cdots + a_n = S_n.$$

The correct habit is to translate everything into a_1, d, n , solve, and then check the requested value.

Another problem uses a geometric progression and asks for a common ratio or a sum. The formula

$$S_n = a_1 \frac{q^n - 1}{q - 1}$$

again appears, with the warning that $q = 1$ is a separate case.

§1.5 Mathematical induction

Induction appears in several distinct forms:

- (i) ordinary induction: prove $P(1)$, then prove $P(n) \Rightarrow P(n + 1)$;
- (ii) strong induction: prove $P(1), \dots, P(n) \Rightarrow P(n + 1)$;
- (iii) induction with a shifted start: begin at $n = m$ instead of 1.

The proof skeleton is:

- (i) base case;
- (ii) induction hypothesis;
- (iii) induction step;
- (iv) conclusion.

The induction hypothesis must be used precisely; one cannot simply assume the desired statement for $n + 1$.

§1.6 Examples of induction

One exercise proves a formula for a sum, such as

$$1 + 2 + \cdots + n = \frac{n(n + 1)}{2}.$$

The induction step is

$$1 + \cdots + n + (n + 1) = \frac{n(n + 1)}{2} + (n + 1) = \frac{(n + 1)(n + 2)}{2}.$$

Another exercise proves a divisibility statement. For such problems, the usual technique is to write the $n + 1$ case in terms of the n case plus a visibly divisible remainder.

The page also contains a problem of the form

$$4^{2k+1} + 3^{k+1}$$

or similar, where the induction step uses modular or factorization identities. The guiding rule is: when proving divisibility by induction, preserve a factor rather than expand everything.

§1.7 Induction structure

A proof by induction has two parts: the base case and the inductive step. The inductive step is not simply checking the next case; it is proving

$$P(n) \Rightarrow P(n+1)$$

for arbitrary n . This is why writing the induction hypothesis explicitly matters.

Strong induction allows the step to use all previous cases:

$$P(1), P(2), \dots, P(n) \Rightarrow P(n+1).$$

It is useful when the object of size $n+1$ breaks into smaller pieces of varying sizes.

§1.8 Modular patterns

When a problem depends on remainders, the natural universe is $\mathbb{Z}/m\mathbb{Z}$. Instead of listing many cases, one should identify the period. If a sequence satisfies

$$a_{n+m} = a_n,$$

then all behavior is determined by the first m terms. This is the organizing idea behind many modular counting exercises.

§1.9 A note on scope

Finite structure links set problems through inclusion–exclusion and modular residues, sequence problems through closed forms, and induction through persistence for all n . The conceptual chain is:

$$\text{example} \Rightarrow \text{pattern} \Rightarrow \text{proof by induction or upper bound}.$$

A mature solution must know which stage it is in. Guessing a pattern is not proving it; constructing an example is not proving optimality; verifying a few cases is not induction.

§1.10 Construction versus upper bound

A combinatorial extremal problem has two halves. A construction proves a lower bound: it shows that at least N objects are possible. An upper bound proves that no construction can exceed N . Only when the two match is the extremal value determined.

Producing a residue-class construction is not the same as proving optimality. The missing step is usually an invariant, counting argument, or compression method that forbids larger examples.

§1.11 Residue classes as finite quotient spaces

When a construction is governed by residues modulo m , the natural universe is

$$\mathbb{Z}/m\mathbb{Z}.$$

A set of integers with forbidden differences becomes a subset of this finite cyclic group with forbidden difference set D . Equivalently, it is an independent set in the Cayley graph whose edges connect residues differing by elements of D .

This transforms a long integer-checking problem into a finite graph problem on residues. The allowed residues are not a random pattern; they form a configuration avoiding forbidden differences in a quotient group.

§1.12 Induction as well-founded recursion

Ordinary induction works because $\mathbb{N}$ is well-ordered. To prove $P(n)$ for all n , it is enough to prove a base case and a step $P(n) \Rightarrow P(n+1)$. Strong induction uses the stronger hypothesis that all smaller cases are known.

The common generalization is well-founded induction. If every nonempty set of objects has a minimal element under a relation $<$, then one may prove $P(x)$ by assuming $P(y)$ for all $y < x$. This is why induction works on integers, trees, expressions, and recursively defined sets.

§1.13 Finite differences in divisibility induction

Many divisibility inductions become cleaner with finite differences. If

$$f(n+1) - f(n)$$

is divisible by m for all n , and $f(n_0)$ is divisible by m , then every later $f(n)$ is divisible by m . This is the discrete fundamental theorem of calculus:

$$f(n) = f(n_0) + \sum_{k=n_0}^{n-1} (f(k+1) - f(k)).$$

So induction is often just summing a difference identity.

§1.14 Floor functions and lattice points

A floor expression records how many integer thresholds have been crossed. Sums such as

$$\sum_{k=0}^{m-1} \left\lfloor \frac{ak+b}{m} \right\rfloor$$

count lattice points under a rational-slope line. This connects floor-function exercises to lattice-point enumeration.

In higher dimensions, Ehrhart theory studies

$$L_P(n) = |nP \cap \mathbb{Z}^d|,$$

the number of lattice points in dilates of a polytope. For integral polytopes, $L_P(n)$ is a polynomial in n . Thus one-variable floor sums are small shadows of a general theory.

§1.15 Pigeonhole and averaging

Many finite counting arguments are averaging arguments. If N objects are distributed among r boxes, then some box has at least

$$\left\lceil \frac{N}{r} \right\rceil$$

objects. This is the quantitative pigeonhole principle.

In extremal problems, one often defines boxes by residues, intervals, or local configurations, then proves that too many objects force a forbidden collision inside one box.

§1.16 Finite structure from quotients and induction

The finite examples are unified by quotienting and persistence. Modular constructions live in finite quotient groups, induction proves persistence, finite differences turn induction into summation, floor functions count lattice points, and extremal constructions require matching lower and upper bounds.